

DIP

Dependency Inversion Principle

Depend on abstractions, not concretions

DEFINITION

High-level and low-level modules both depend on abstractions, not on each other; the abstraction doesn't depend on details — details implement it.

"Inversion" flips the usual import direction. Normally high-level code imports the database and SDKs it uses; DIP says define an abstraction (a port) in the domain layer and have those low-level details implement it. Source-code dependencies then point inward, toward policy — the details depend on the domain, not the other way around.

WHY IT MATTERS

When business logic imports a concrete database or SDK, you cannot test it without that infrastructure and cannot swap the detail without editing the core. Inverting the dependency lets the domain be tested with a fake and lets infrastructure change (Postgres to DynamoDB) without touching a single business rule. It is the principle that makes hexagonal and clean architecture possible.

— How it works

Port

An abstraction owned by the domain layer (OrderRepo, Clock, Mailer). It states what policy needs, in policy's own terms.

Adapter

A low-level implementation of the port (SqlOrderRepo, SystemClock). It lives in the infrastructure layer and depends on the domain, not vice versa.

Composition root

The one outer place that wires concrete adapters into policy (a DI container or main). The domain never names a concretion.

HOW TO APPLY

1. Spot where the domain reaches for infrastructure — a DB client, an SDK, the clock.
2. Define a port in the domain that expresses the need in domain terms.
3. Implement that port with an adapter in the infrastructure layer.
4. Wire the adapter to policy at the composition root, so the domain depends only on the port.

LITMUS TEST

Does your domain layer import anything from the database, HTTP, or framework? If yes, the dependency runs the wrong way.

— Smell → fix

The service reaches straight for a concrete Postgres client:

```
import { PostgresClient } from './pg'
class OrderService {
  private db = new PostgresClient() // domain bound to infra
  save(o) { this.db.insert('orders', o) }
}
```

↓ INVERT VIA A DOMAIN-OWNED PORT

```
interface OrderRepo { save(o: Order): Promise }
function makeOrderService(repo: OrderRepo) { // depends on the port
  return { place(o) { return repo.save(o) } }
}
class SqlOrderRepo implements OrderRepo { save(o){ /* ... */ } }
```

The service depends only on OrderRepo, a port owned by the domain. The concrete SqlOrderRepo implements it and is supplied from outside (DI).

USE WHEN

- ✓ Keep the domain free of DB/framework imports
- ✓ Swap implementations, test in isolation

AVOID WHEN

- ▲ Trivial code with a single, stable implementation

— Trade-offs & pitfalls

PROS

- + Testable, swappable
- + Clean domain core

CONS

- More wiring
- Needs a DI mechanism/container

COMMON PITFALLS

- ▲ Calling field or parameter injection "DIP" while the domain still imports the concrete class — the dependency hasn't actually inverted.
- ▲ A port that leaks infrastructure terms (SQL, HTTP status) — the abstraction must speak the domain's language, not the detail's.
- ▲ Abstracting a single, stable implementation that will never be swapped — interfaces as ceremony (YAGNI).

WHERE YOU SEE IT

- ▶ NestJS providers: a service depends on an injected REPOSITORY token; the SQL implementation is bound in the module.
- ▶ Hexagonal architecture: domain ports with infrastructure adapters around the edge.
- ▶ Passing a Clock or Logger abstraction so tests inject a fake instead of the real one.

SEE ALSO

Dependency Injection the mechanism that supplies the concrete implementation at runtime

Repository a common port DIP defines over persistence

OCP depending on abstractions is what makes modules extension-friendly

SoC DIP keeps the domain concern free of infrastructure concerns

— Interview & sources

DIP vs Dependency Injection — same thing?

No. DIP is the principle (depend on abstractions, invert the import direction). DI — supplying dependencies from outside via a container or factory — is one mechanism that delivers it.

What does the abstraction belong to?

The high-level/domain layer owns the interface; the low-level module implements it. That ownership is what makes it "inversion".

When is DIP overkill?

For trivial code with a single stable implementation an interface is just ceremony. Invert where you need testability or substitution.

DIP, DI and IoC — how do they fit?

IoC is the broad idea that a framework calls you; DI is one form of IoC that supplies dependencies from outside; DIP is the design rule that those dependencies be abstractions the high-level layer owns.

REMEMBER

High-level policy and low-level detail both depend on an abstraction the policy owns.

SOURCES

Robert C. Martin — "The Dependency Inversion Principle" (C++ Report, 1996)

Robert C. Martin — "Clean Architecture" (2017): the dependency rule